```
3 4 5
4 5 6
5 6 7
6 7 8
7 8 9
8 9 10
```

It's quite common to need to hold on to what the index of each successive element is in the block. You can do this using each_with_index, which takes a two-element block, the second element of which is the index, starting at zero:

```
elements = ["a", "b", "c", "d"]
elements.each_with_index { |x, i| puts "#{x} is at index #{i}" }
```

produces:

```
a is at index 0
b is at index 1
c is at index 2
d is at index 3
```

Accessing

In general, the Enumerable interface is not used for arbitrary access—subclasses like Array and Hash typically override the square bracket operator to enable this. However, a couple of Enumerable methods do allow for access. The simplest is to_a (aliased to entries), which returns an array of the successive elements in the container.

You can get the first elements of an Enumerable with take, which takes one argument and returns that many elements from the start of the array. There's also first, which behaves the same except you can call first with no arguments to get the single first element.

On the other side, the drop method takes an argument and returns all the elements after that many arguments from the start of the array:

```
ex = ["a", "b", "c", "d", "e", "f", "g"]
ex.first    # => "a"
ex.take(2)  # => ["a", "b"]
ex.drop(2)  # => ["c", "d", "e", "f", "g"]
```

Both take and drop have alternate forms: take_while and drop_while. Each method takes a block and applies the block to successive elements of the array until the block returns a falsey value. At that point, take_while returns all the previously processed elements, while drop_while returns the failing element and all subsequent elements. For example:

```
ex = [2, 4, 8, 16, 32, 64]
ex.take_while { |x| x < 10 } # => [2, 4, 8]
ex.drop_while { |x| x < 10 } # => [16, 32, 64]
```

Map and Reduce

A common pattern when dealing with data in containers is to process the data in two steps: a *map* step, where the each element of the container is transformed, and a *reduce* step, where the entire container is combined to a single value. At large scale, the map-reduce pattern is a way to structure big data manipulations such that they can be easily parallelized. At